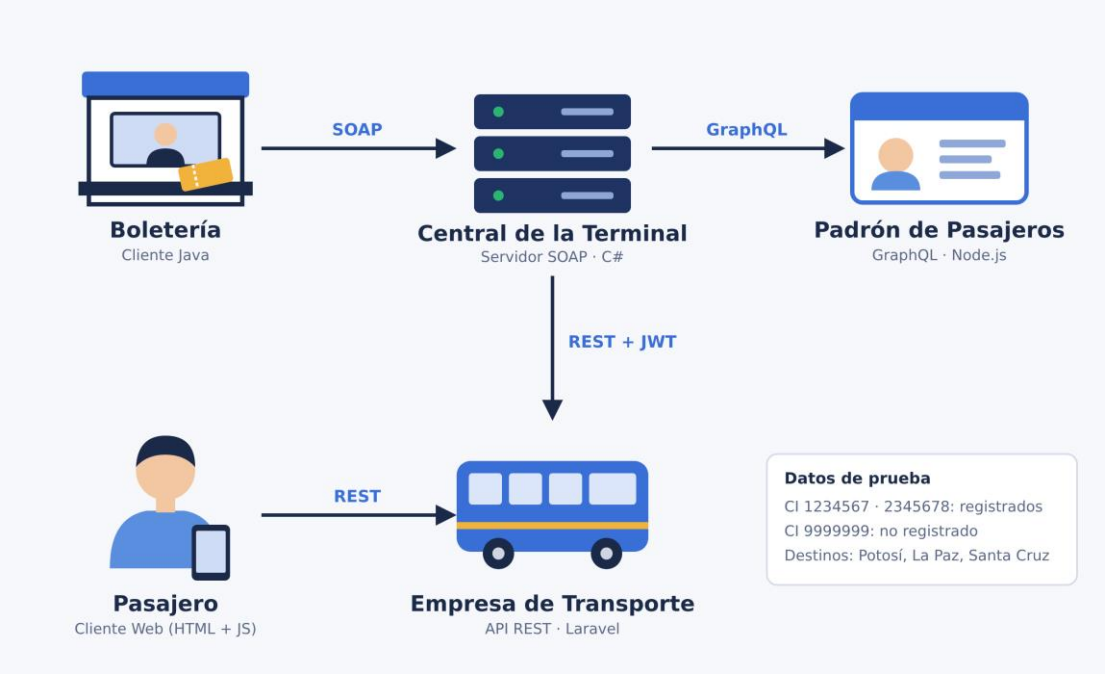
Carrera Ingeniería de Sistemas
Segundo Parcial Sistemas Distribuidos SIS-258 Práctico

Fecha: …… de noviembre de 2026 2/2026	Nombre y Apellidos: …… …… Nro en Lista:….

IV. Parte Práctica (70 puntos)

Sistema de Venta de Pasajes Interdepartamentales

La **Terminal de Buses de Sucre** ha decidido implementar un **Sistema Distribuido de Venta de Pasajes Interdepartamentales**, que permita vender boletos en las boleterías de la terminal y consultar las salidas por Internet, integrando la información de las **empresas de transporte** y del **Padrón de Pasajeros** de la terminal. Cada componente se desarrolla en un lenguaje distinto y todos se comunican mediante servicios web **SOAP, REST y GraphQL**, de acuerdo al siguiente esquema:



El sistema está compuesto por cinco componentes, cada uno desarrollado en una tecnología distinta. Se debe construir y conectar:

1. **Padrón de Pasajeros — GraphQL en Node.js (15 puntos)**

La terminal mantiene un padrón de los pasajeros habilitados para viajar, expuesto mediante un **servicio GraphQL**.

- Base de datos con la tabla *pasajeros*: CI, nombres, apellidos, celular y si es pasajero frecuente.
- Query *pasajero(ci)* que devuelve los datos del pasajero y query *pasajeros* que devuelve la lista completa.

- Mutation *registrarPasajero(ci, nombres, apellidos, celular)* que valide que el CI tenga entre 5 y 10 dígitos y que no esté registrado previamente.
- Si el pasajero no existe o los datos son inválidos, el error debe llegar en el campo *errors* de la respuesta con un mensaje claro (por ejemplo, "Pasajero no registrado").

2. **API de la Empresa de Transporte — REST en Laravel (20 puntos)**

Cada empresa de transporte publica sus salidas y la ocupación de sus buses mediante un **servicio REST** con autenticación **JWT**. Los recursos se llaman *salidas* (destino, fecha, hora, bus, precio y capacidad) y *asientos* (salida, número de asiento y CI del pasajero).

- *POST /api/login*: recibe email y contraseña y devuelve un token JWT; responde *401* si las credenciales son inválidas.
- *GET /api/salidas?destino=&fecha=* (público): lista las salidas filtradas, indicando la cantidad de asientos libres de cada una.
- *GET /api/salidas/{id}* (público): devuelve la salida con sus asientos ocupados; *404* si no existe.
- *POST /api/salidas* (requiere token): registra una nueva salida; *201* si se crea y *422* si los datos son inválidos.
- *PUT /api/salidas/{id}/asientos/{numero}* (requiere token), con el CI del pasajero en el cuerpo JSON: reserva el asiento. Responde *200* si se reserva, *404* si la salida no existe, *409* si el asiento ya está ocupado y *422* si el número de asiento excede la capacidad del bus o el CI es inválido.

3. **Central de la Terminal — SOAP en C# (20 puntos)**

La Central es un **servicio web SOAP** que la boletería utiliza para vender boletos. Es **servidor** para la boletería y, al mismo tiempo, **cliente** del Padrón (GraphQL) y de la Empresa de Transporte (REST). Debe exponer las operaciones:

- *Salida[] VerSalidas(String destino, String fecha)*: consulta las salidas en la API REST de la empresa y las devuelve como un arreglo de objetos.
- *Boleto VenderBoleto(String ci, int idSalida, int asiento)*: verifica al pasajero en el Padrón mediante una consulta **GraphQL con variables**, se autentica en la API REST, reserva el asiento enviando el token en la cabecera *Authorization: Bearer <token>* y devuelve un objeto *Boleto* con código, nombre del pasajero, destino, fecha, hora, número de asiento y precio.
- Si el pasajero no está registrado, el asiento está ocupado (409) o los datos son inválidos, debe lanzar un **Fault** de tipo *Client*; si alguno de los servicios no responde, un **Fault** de tipo *Server*, ambos con un mensaje claro.

4. **Clientes — acceso desde distintos lenguajes (15 puntos)**
   - **Boletería (Java, escritorio):** cliente que consume el servicio SOAP de la Central a partir de su **WSDL** (clases generadas con wsimport). Permite buscar salidas por destino y fecha, vender un boleto eligiendo la salida y el asiento, y mostrar el comprobante. Si la venta falla, muestra el mensaje del Fault. (8 puntos)
   - **Pasajero (Cliente Web, HTML + JavaScript):** página que consume **directamente la API REST** de la empresa mediante *fetch* para consultar las salidas disponibles por destino y fecha y ver los asientos libres de una salida, mostrando los mensajes de error que devuelva la API. (7 puntos)

Para SOAP utilizar C# (ASMX)
Para REST Laravel
Para GraphQL Node.js
Para los clientes: Java (Boletería) y HTML + JavaScript (Pasajero)